

Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

Word Embedding

(Non-Examinable Supplemental Material)

François Pitié

Assistant Professor in Media Signal Processing
Department of Electronic & Electrical Engineering, Trinity College Dublin

[4C16/5C16] Deep Learning and its Applications — 2026/2027

At the core of neural networks is the use of linear algebra to combine feature vectors. Using neural networks for text processing is therefore problematic, as text is a sequence of words and symbols rather than a vector of real numbers.

In this handout, we will look at how text can be represented with real-valued **word vectors**. This transformation is a key step in using machine learning for **natural language processing** (NLP) and is called encoding or embedding.

The aim of an embedding is to design a lookup table, where each word can be mapped to a vector.

One-Hot Encoding

The simplest embedding is the **one-hot** vector. Each symbol in a dictionary of size n is represented as a vector where all elements are zero, except for a single entry of one at the index corresponding to that word.

Example

Consider the following alphabetically sorted dictionary: `{a, cat, is, sitting}`. The dictionary size is 4, so the word vector sizes are 4×1. The one-hot encodings for `cat` and `sitting` could be as follows:

$$\mathbf{x}_{\texttt{cat}} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{x}_{\texttt{sitting}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

One-Hot Encoding

Note that **one-hot** encoding can be used to encode more than just words and can be used to encode sequences of any kind of symbols.

Example
For instance, say we want to encode the 26 letters from the alphabet. Each character can be represented as follows:

$$\mathbf{x}_{\mathtt{a}} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad \mathbf{x}_{\mathtt{b}} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ \vdots \end{bmatrix}, \quad \mathbf{x}_{\mathtt{c}} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \quad \cdots$$

One-Hot Encoding

The problem with one-hot word vectors is the sheer scale of languages. There are about 200,000 words in current use in the English language, which altogether create about 13 million different tokens. One word (*e.g.* `have`) can have many tokens (*e.g.* `having`, `had`, `has`, `'s`). Word vectors generated by one-hot encoding would be embedded in a vector space of dimension 13 million!

In neural networks, this would require input vectors of dimension 13 million and thus would necessitate training an enormous number of weights.

We therefore need to find a more compact representation.

One-Hot Encoding

Another issue is that one-hot encoding does not capture any semantic relationship between words.

Let's look at the word vectors we have defined with one-hot. In theory we could add the word vectors as follows:

$$2\mathbf{x}_{\texttt{cat}} + 3.2\mathbf{x}_{\texttt{sitting}} - 1.5\mathbf{x}_{\texttt{is}}$$

but we will end up with vectors that are difficult to interpret. For instance, what would be the meaning of

$$2\mathbf{x}_{\texttt{cat}} + 3.2\mathbf{x}_{\texttt{sitting}} - 1.5\mathbf{x}_{\texttt{is}} = \begin{bmatrix} 0 \\ 2 \\ -1.5 \\ 3.2 \end{bmatrix} = \mathbf{x}_?$$

One-Hot Encoding

Also, by definition, for any two distinct dictionary entries $\mathbf{x}_1$ and $\mathbf{x}_2$, we have $\mathbf{x}_1^\top \mathbf{x}_2 = 0$. Consequently, their **cosine similarity**

$$\text{similarity}(\mathbf{x}_1, \mathbf{x}_2) = \frac{\mathbf{x}_1^\top \mathbf{x}_2}{\|\mathbf{x}_1\| \, \|\mathbf{x}_2\|}$$

is always zero, regardless of whether the words are synonyms or entirely unrelated.

We require an encoding where words are represented by vectors that make sense in a **Euclidean space**: where linear combinations of word vectors and scalar products make as much sense as possible.

Word2Vec

Word2Vec (2013) was the first neural embedding model to gain widespread popularity, though many competing non-deep learning techniques exist—such as **GloVe** (Global Vectors for Word Representation), which relies on matrix factorisation of co-occurrence statistics.

Word2Vec proposes two main predictive approaches:

- **Continuous Bag of Words** (CBOW), in which the context is used to predict a target word;
- **Skip-gram**, in which the context is predicted from a target word.

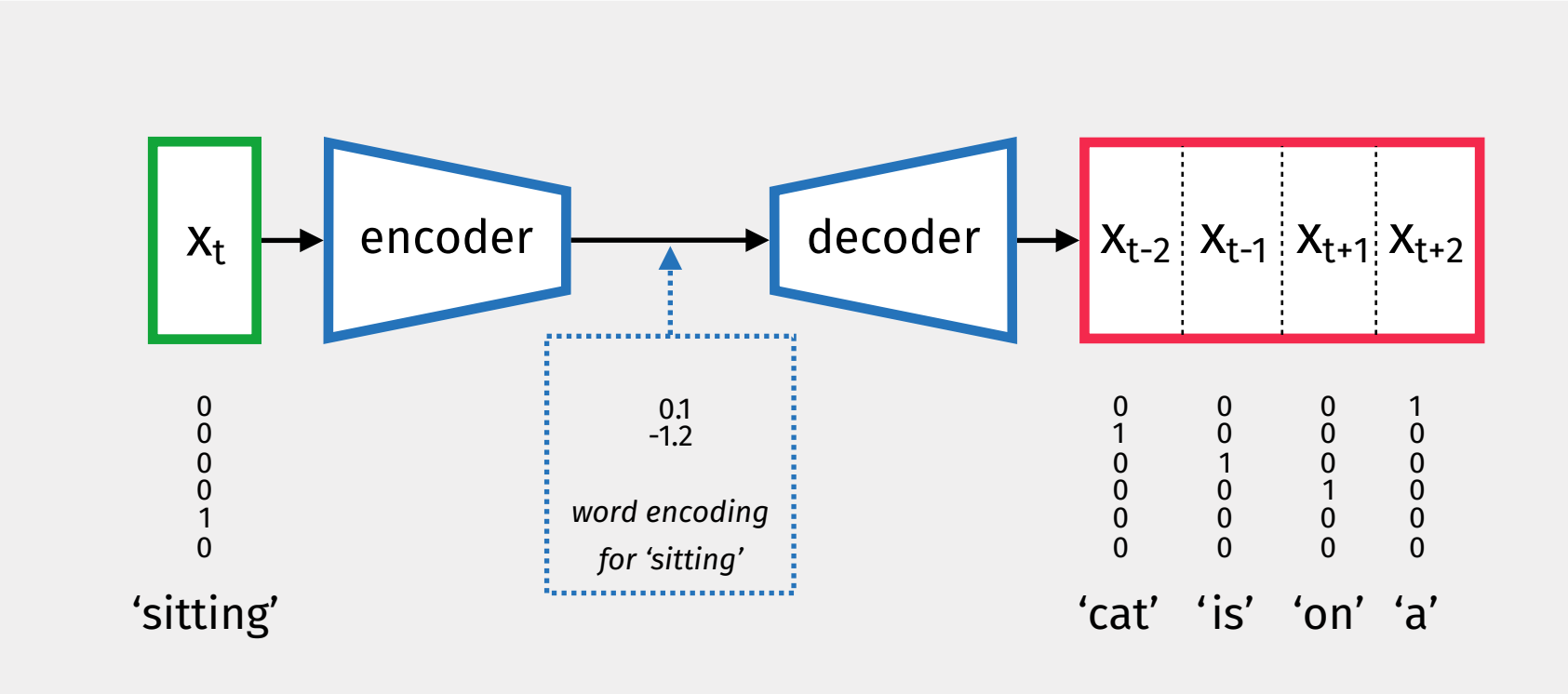
Word2Vec: Skip-gram

Let's look at the Skip-gram approach, where we take a target word $\mathbf{x}_t$ and attempt to predict its surrounding context words (*e.g.* $\mathbf{x}_{t-2}$, $\mathbf{x}_{t-1}$, $\mathbf{x}_{t+1}$, $\mathbf{x}_{t+2}$).



In the Skip-gram approach of Word2Vec, the input is $\mathbf{x}_t$ and we are trying to predict the 4 context vectors $[\mathbf{x}_{t-2}, \mathbf{x}_{t-1}, \mathbf{x}_{t+1}, \mathbf{x}_{t+2}]$.

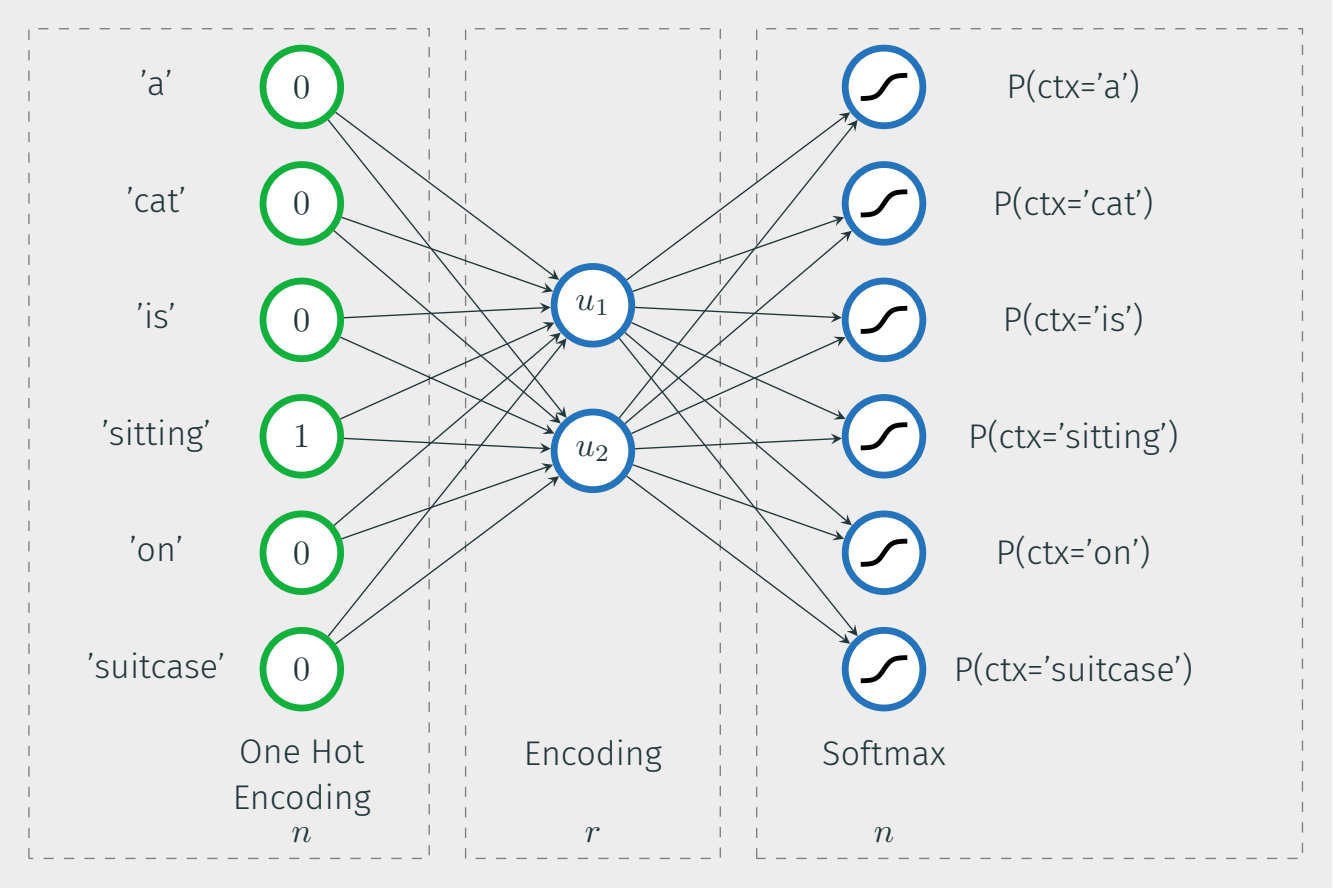
Word2Vec: Architecture



The model follows an encoder-decoder architecture. The input is a one-hot vector of size n (the size of the dictionary), the encoder produces a compact embedding of size r, and the decoder attempts to reconstruct the context word probabilities.

Word2Vec: Network Detail

In detail, the encoder-decoder networks are very simple. Below is an example for a single context word prediction. The encoder only consists of a single dense layer of size r with linear activations. The decoding layer is simply a softmax layer of size n.



Word2Vec

Note that training this model is non-trivial as the final softmax layer must handle millions of entries.

Calculating the exponentials for such a large dictionary is computationally complex, requiring specialised techniques such as **hierarchical softmax** or **negative sampling**.

Doing Maths with these Embeddings

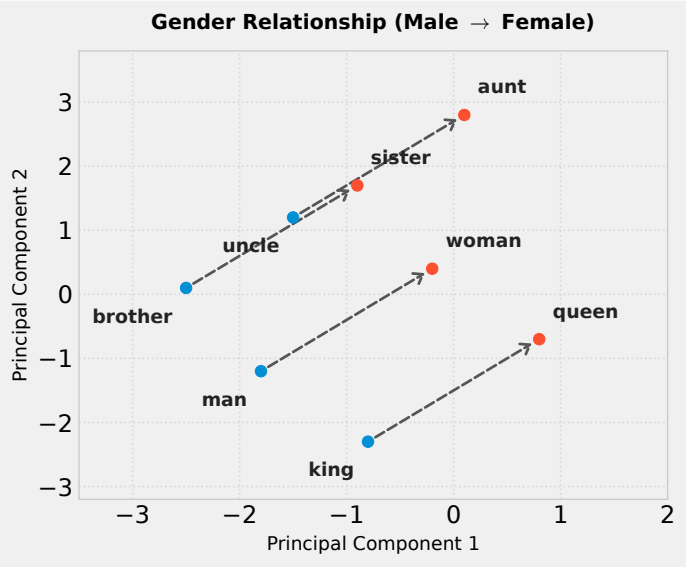
After training, we observe that the learnt embedding spaces capture semantic and syntactic relationships. Notably we observe **linear substructures**, which allow for algebraic manipulation of concepts through simple vector arithmetic.

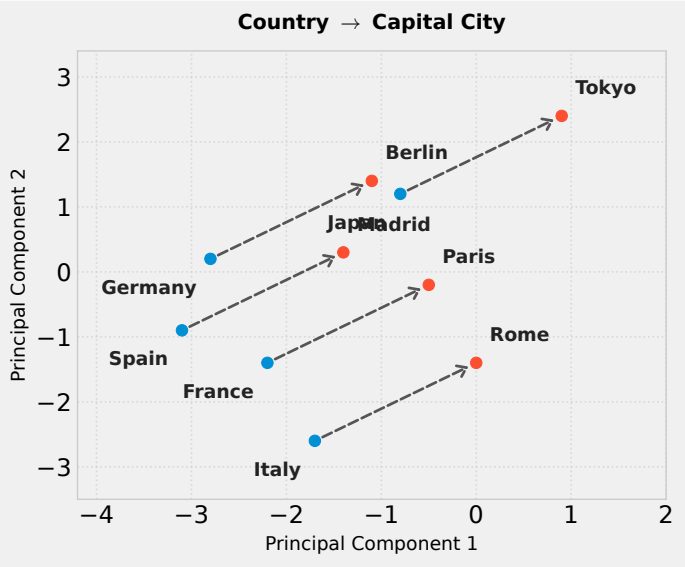
A famous example is the gender / royalty relationship demonstrated by Mikolov *et al.* (2013):

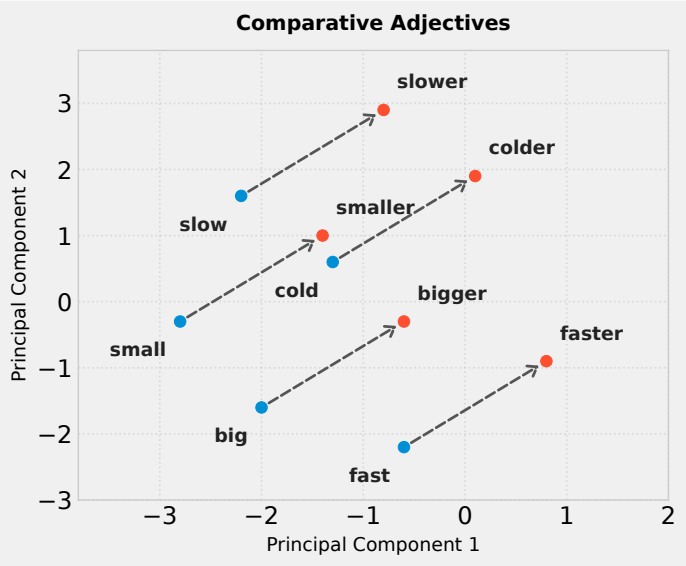
$$\mathbf{x}_{\text{king}} - \mathbf{x}_{\text{man}} + \mathbf{x}_{\text{woman}} \approx \mathbf{x}_{\text{queen}}$$

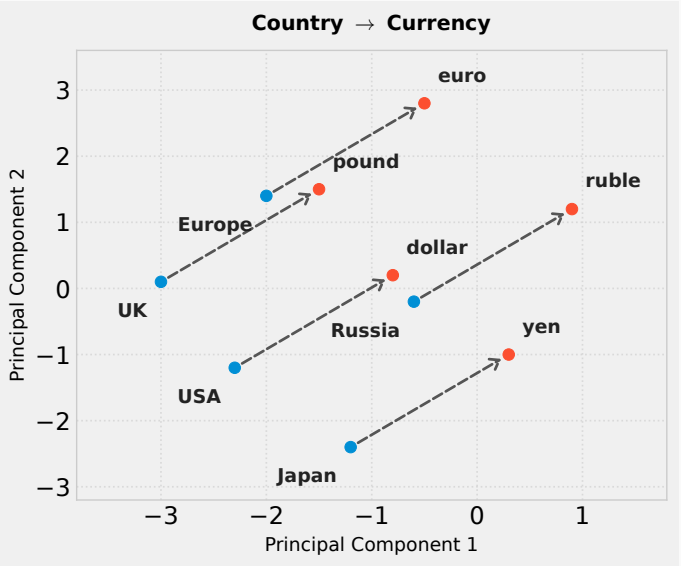
When projected onto a two-dimensional subspace (*e.g.* via Principal Component Analysis), word pairs sharing the same relationship are connected by approximately parallel translation vectors.

Linear Substructures in Word2Vec









Additive Compositionality

Similarly, Mikolov *et al.* (2013) discovered that element-wise vector addition acts a bit like a semantic "AND" operator:

Vector Addition	Closest Tokens in Word2Vec Space
$\mathbf{x}_{\text{Czech}} + \mathbf{x}_{\text{currency}}$	`koruna`, `Czech crown`, `Polish zloty`
$\mathbf{x}_{\text{Vietnam}} + \mathbf{x}_{\text{capital}}$	`Hanoi`, `Ho Chi Minh City`, `Vietnamese`
$\mathbf{x}_{\text{German}} + \mathbf{x}_{\text{airlines}}$	`Lufthansa`, `airline`, `German Airways`
$\mathbf{x}_{\text{Russian}} + \mathbf{x}_{\text{river}}$	`Volga River`, `upriver`, `Dnieper`

Nearest Neighbours and Semantic Similarity

These embeddings are equally effective at capturing semantic proximity using the **cosine similarity** (scalar product between normalised vectors):

$$\text{similarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|}$$

For instance, computing the nearest neighbours to the query word `"frog"` in the pre-trained Google News Word2Vec model yields closely related amphibians:

Rank	Nearest Neighbour	Cosine Similarity
1	`frogs`	0.80
2	`toad`	0.79
3	`bullfrog`	0.73
4	`tree_frog`	0.70
5	`toads`	0.69

Tokenisation: Beyond Word-Level Embeddings

While word-level embeddings like Word2Vec and GloVe show that text can be meaningfully mapped into a Euclidean space, they suffer from significant limitations:

1. **Out-of-Dictionary (OOD) Problem:** If a model encounters a word that was not in its training vocabulary (*e.g.* a typo, a new technical term, or a rare name), it cannot generate a vector for it.
2. **Morphology:** They treat different forms of the same root word (*e.g.* "walk", "walking", "walked") as entirely distinct.

Tokenisation: Byte-Pair Encoding & WordPiece

Modern Large Language Models (LLMs) solve these issues by using tokenisation instead of simple word-level encoding. Techniques such as **Byte-Pair Encoding (BPE)** or **WordPiece** break down words into frequent groups of bytes or sub-word units.

Common words remain as single tokens, while rare words are decomposed into smaller, more frequent sub-units (*e.g.* "unhappily" → **un** + `happi` + `ly`).

This offers a bridge between character-level and word-level encoding:

- Like character-level encoding, it can represent any arbitrary string (even down to individual bytes), eliminating the out-of-dictionary problem.
- Like word-level encoding, it retains semantic meaning in its tokens, as frequent sub-words (prefixes/suffixes) carry consistent information.

Do LLM Tokens Exhibit Linear Substructures?

Do we observe similar linear relationships among tokens in modern LLMs? The short answer is **no, not at the token embedding level**.

This is partly because tokens are often morphological fragments (such as `" un"`, `"token"`, or `"ization"`). Summing sub-word embeddings does not produce the embedding of the compound word.

Linear Representation Hypothesis (LRH)

Linear substructures are, however, found in the depth of the LLM. Mechanistic interpretability research has shown that linear structure migrates from the input token table into the *residual stream*, the layer-by-layer communication bus that connects all Transformer blocks.

There, just as with Word2Vec, we can perform *representation engineering* by adding a constant steering vector $\mathbf{v}_{\text{concept}}$ directly into the residual stream at test time:

$$\mathbf{x}_l \leftarrow \mathbf{x}_l + \alpha \mathbf{v}_{\text{concept}}$$

predictably shifting the model's tone, reasoning, or honesty without fine-tuning weights.

Thus, the fundamental geometric intuition—that meaning can be organised along linear directions—remains central to how deep networks reason.

Further Reading

[Word2Vec] Efficient Estimation of Word Representations in Vector Space
T. Mikolov, K. Chen, G. Corrado, J. Dean, 2013 [https://arxiv.org/abs/1301.3781]

[GloVe] GloVe: Global Vectors for Word Representation
J. Pennington, R. Socher, C. Manning, 2014 [https://nlp.stanford.edu/projects/glove/]

[BPE] Neural Machine Translation of Rare Words with Subword Units
R. Sennrich, B. Haddow, A. Birch, 2015 [https://arxiv.org/abs/1508.07909]

[Circuits] A Mathematical Framework for Transformer Circuits
C. Elhage et al., 2021 [https://transformer-circuits.pub/2021/framework/index.html]

[Truth] The Geometry of Truth: Emergent Linear Structure in LLM Representations
S. Marks, M. Tegmark, 2023 [https://arxiv.org/abs/2310.06824]

[LRH] The Linear Representation Hypothesis and the Geometry of Concepts
K. Park et al., 2023 [https://arxiv.org/abs/2311.03658]